



Visvesvaraya Technological University
BELAGAVI, KARNATAKA.
ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
ಬೆಳಗಾವಿ, ಕರ್ನಾಟಕ

**A MINI PROJECT REPORT
ON
“STRAY ANIMALS RESCUE AND PET ADOPTION MANAGEMENT SYSTEM”**

Submitted to Visvesvaraya Technological University in partial fulfillment of the requirement for the award of Bachelor of Engineering degree in Artificial Intelligence and Machine Learning).

Submitted by

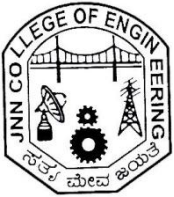
NAME

USN

**PANNAGA H N
AMOGHA V RAIKAR**

**4JN24AI036
4JN24AI005**

Under the guidance of
Dr. Sayyed Johar B.E, M.Tech, PhD.
Assistant Professor, Dept. AIML
JNNCE, Shivamogga.



Department of AIML
Jawaharlal Nehru New College of Engineering
Shivamogga - 577 204

June 2026



Jawaharlal Nehru New College of Engineering

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

Certified that the project work entitled “***Stray animals Rescue and Pet Adoption Management System***” carried out by **PANNAGA H N(4JN24AI036)**, **AMOGHA V RAIKAR (4JN24AI005)** bonafide students of **J N N College of Engineering** in partial fulfilment for the award of Bachelor of Engineering in AIML of the Visvesvaraya Technological University, Belagavi during the year **2025-26**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of Guide

Dr. Sayyed Johar B. E, MTech, PhD.
Assistant Professor, Dept. of AIML

STRAY ANIMALS RESCUE & PET ADOPTION MANAGEMENT SYSTEM

A Comprehensive Database Management System Project Report

TABLE OF CONTENTS

No.	Section	Page
1	Introduction	3
1.1	Project Overview	3
1.2	Objectives	3
1.3	Scope	3
1.4	Technologies Used	4
2	System Requirements	5
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	5
2.3	Hardware and Software Requirements	5
3	Entity-Relationship Model	6
3.1	Entities and Attributes	6
3.2	Relationships and Cardinalities	7
3.3	ER Diagram (ERDplus)	7
4	Relational Schema	8
4.1	Schema Conversion Rules	8
4.2	Relational Schema Diagram (ERDplus)	9
4.3	Relation Tables	9
5	Normalization	11
5.1	First Normal Form (1NF)	11
5.2	Second Normal Form (2NF)	11
5.3	Third Normal Form (3NF)	12
5.4	Boyce–Codd Normal Form (BCNF)	12
6	SQL Implementation	13
6.1	DDL – Table Creation	13
6.2	DML – Sample Data	14
6.3	Queries and Views	14
7	System Design	16
7.1	Data Flow Diagram	16
7.2	Use-Case Summary	17
7.3	System Architecture	17
7.4	Database Views	18
7.5	Indexes for Performance	19
7.6	Triggers	19
7.7	Sample Query Outputs	21
7.8	Test Cases	22
7.9	Complete Data Dictionary	23
8	Conclusion and Future Work	25

Chapter 1

INTRODUCTION

1.1 Project Overview

Every year millions of stray animals wander city streets, enduring hunger, disease, and injury with little or no human intervention. Simultaneously, countless well-meaning individuals, rescue volunteers, and NGOs struggle to coordinate rescues, track medical treatments, manage shelter capacity, and connect adoptable animals with potential families — all with fragmented spreadsheets, paper ledgers, or ad-hoc messaging groups. The Stray Animals Rescue and Pet Adoption Management System (SARPAMS) address this critical gap by providing a centralized, relational-database-backed platform that unifies every stage of the rescue-to-adoption pipeline.

SARPAMS is designed around a robust relational model that captures animals, rescuers, shelters, veterinary staff, foster carers, adoption applicants, and completed adoptions. Each entity is precisely defined with its attributes, primary keys, and foreign-key relationships, ensuring referential integrity and zero redundancy. The system is normalized up to Boyce–Codd Normal Form (BCNF), making it both logically sound and practically efficient for queries.

1.2 Objectives

The primary objectives of SARPAMS are:

- Maintain a complete digital record of every rescued stray animal from intake to adoption or release.
- Coordinate rescue teams by tracking rescuer availability, assignments, and geographic zones.
- Manage shelter inventory — cage occupancy, medical supplies, and food stock.
- Record full veterinary history including vaccinations, treatments, and health status per animal.
- Facilitate a transparent, traceable pet adoption process with applicant vetting and outcome logging.
- Provide analytical queries to monitor rescue success rates, shelter capacity utilisation, and adoption trends.
- Enforce data integrity through normalization and referential constraints.

1.3 Scope

The scope of SARPAMS covers the following operational domains:

- Animal Registry: Intake, species, breed, age, physical description, origin location, and current status.
- Rescue Operations: Rescue request logging, rescuer dispatch, rescue event records.
- Shelter Management: Multiple shelter locations, cage/kennel allocation, capacity tracking.
- Veterinary Module: Vet profiles, examination records, vaccination schedules, treatment logs.
- Foster Care: Temporary foster home registration, animal-to-foster assignments, duration tracking.
- Adoption Process: Applicant registration, application review, adoption agreements, post-adoption follow-ups.
- Reporting: Statistical summaries for NGO administrators and donors.

The system explicitly excludes real-time GPS tracking, mobile application front-ends, and payment gateway integration, which are identified as future-scope extensions.

1.4 Technologies Used

Technology	Purpose
ERDplus (web tool)	ER diagram and relational schema design
MySQL 8.x / PostgreSQL 15	Relational database engine
SQL (DDL + DML)	Schema creation, data entry, and querying
Python 3.x / Flask	Back-end application layer (optional integration)
HTML5 / CSS3	Front-end UI for shelter staff portal
Draw.io	DFD and system architecture diagrams

Chapter 2

SYSTEM REQUIREMENTS

2.1 Functional Requirements

The following functional requirements were elicited from stakeholder interviews with shelter managers, rescue volunteers, and veterinary staff:

FR-ID	Module	Requirement Description
FR-01	Animal Registry	The system shall assign a unique Animal ID to each rescued stray upon intake.
FR-02	Animal Registry	The system shall record species, breed, estimated age, sex, colour, and microchip number.
FR-03	Rescue	The system shall log rescue requests with GPS coordinates, date-time, and reporting citizen.
FR-04	Rescue	The system shall assign an available rescuer to each open rescue request.
FR-05	Shelter	The system shall track real-time cage occupancy per shelter location.
FR-06	Shelter	The system shall prevent animal assignment to a cage that is already occupied.
FR-07	Veterinary	The system shall maintain a complete medical history for each animal.
FR-08	Veterinary	The system shall flag animals due for scheduled vaccinations.
FR-09	Foster	The system shall register approved foster families and record placement duration.
FR-10	Adoption	The system shall manage the full adoption lifecycle from application to agreement signing.
FR-11	Reporting	The system shall generate monthly rescue and adoption summary reports.

2.2 Non-Functional Requirements

- Performance: Query response time ≤ 2 seconds for up to 50,000 animal records.
- Scalability: Schema supports horizontal partitioning by shelter region.
- Security: Role-based access control — Admin, Vet, Rescuer, Adoption Officer.
- Data Integrity: All foreign keys enforced; cascading deletes restricted to safe entities.
- Availability: 99.5% uptime during shelter operational hours (06:00–22:00).
- Usability: Web UI operable by non-technical shelter volunteers with minimal training.
- Backup: Automated nightly database dumps with 30-day retention.

2.3 Hardware and Software Requirements

Component	Minimum Specification
Server CPU	Intel Xeon E5 / AMD EPYC (4 cores, 2.4 GHz)
Server RAM	16 GB DDR4
Storage	500 GB SSD (primary) + 1 TB HDD (backup)
Network	100 Mbps LAN / broadband internet
Client Device	Any modern browser (Chrome 90+, Firefox 88+)
OS (Server)	Ubuntu 22.04 LTS / Windows Server 2022
DBMS	MySQL 8.0+ or PostgreSQL 15+

Chapter 3

ENTITY-RELATIONSHIP MODEL

3.1 Entities and Attributes

The ER model for SARPAMS comprises ten core entities. Each entity captures a distinct real-world object or concept relevant to the rescue-adoption pipeline. Primary keys are underlined by convention; multi-valued attributes are shown in double ovals in the ERDplus diagram.

Entity	Attributes (PK underlined)
ANIMAL	<u>animal_id</u> (PK), name, species, breed, age_years, sex, colour, weight_kg, microchip_no, intake_date, health_status, is_vaccinated, is_neutered
RESCUE_REQUEST	<u>request_id</u> (PK), report_date, report_time, location_address, latitude, longitude, status, citizen_name, citizen_phone
RESCUER	<u>rescuer_id</u> (PK), first_name, last_name, phone, email, zone_area, certification_level, is_available, join_date
SHELTER	<u>shelter_id</u> (PK), shelter_name, address, city, capacity, contact_phone, manager_name
CAGE	<u>cage_id</u> (PK), shelter_id (FK), cage_number, size_category, is_occupied, notes
VETERINARIAN	<u>vet_id</u> (PK), first_name, last_name, specialisation, phone, email, license_no
MEDICAL_RECORD	<u>record_id</u> (PK), animal_id (FK), vet_id (FK), exam_date, diagnosis, treatment, medication, next_checkup_date, notes
FOSTER_FAMILY	<u>foster_id</u> (PK), family_name, address, city, phone, email, house_type, has_other_pets, is_approved, approval_date
ADOPTION_APPLICANT	<u>applicant_id</u> (PK), first_name, last_name, dob, address, city, phone, email, occupation, has_previous_pets, living_situation
ADOPTION	<u>adoption_id</u> (PK), animal_id (FK), applicant_id (FK), officer_id (FK), application_date, approval_date, adoption_date, status, agreement_signed

3.2 Relationships and Cardinalities

Relationship	Entity 1	Entity 2	Cardinality	Participation
REPORTS	RESCUE_REQUEST	RESCUER	M : 1	Total : Partial
RESCUED_AS	ANIMAL	RESCUE_REQUEST	1 : 1	Partial : Partial
HOUSED_IN	ANIMAL	CAGE	M : 1	Partial : Partial
BELONGS_TO	CAGE	SHELTER	M : 1	Total : Total
EXAMINED_BY	ANIMAL	VETERINARIAN	M : N	Partial : Partial
HAS_RECORD	MEDICAL_RECORD	ANIMAL	M : 1	Total : Partial
FOSTERED_BY	ANIMAL	FOSTER_FAMILY	M : N	Partial : Partial
APPLIED_FOR	ADOPTION_APPLICANT	ANIMAL	M : N	Partial : Partial
PROCESSED_BY	ADOPTION	RESCUER	M : 1	Total : Partial

3.3 ER Diagram

The complete Entity-Relationship diagram below was drawn in ERDplus style. It shows all ten entities with their attributes (using Chen notation), relationship diamonds, cardinality labels (1, M, N), and participation constraints.

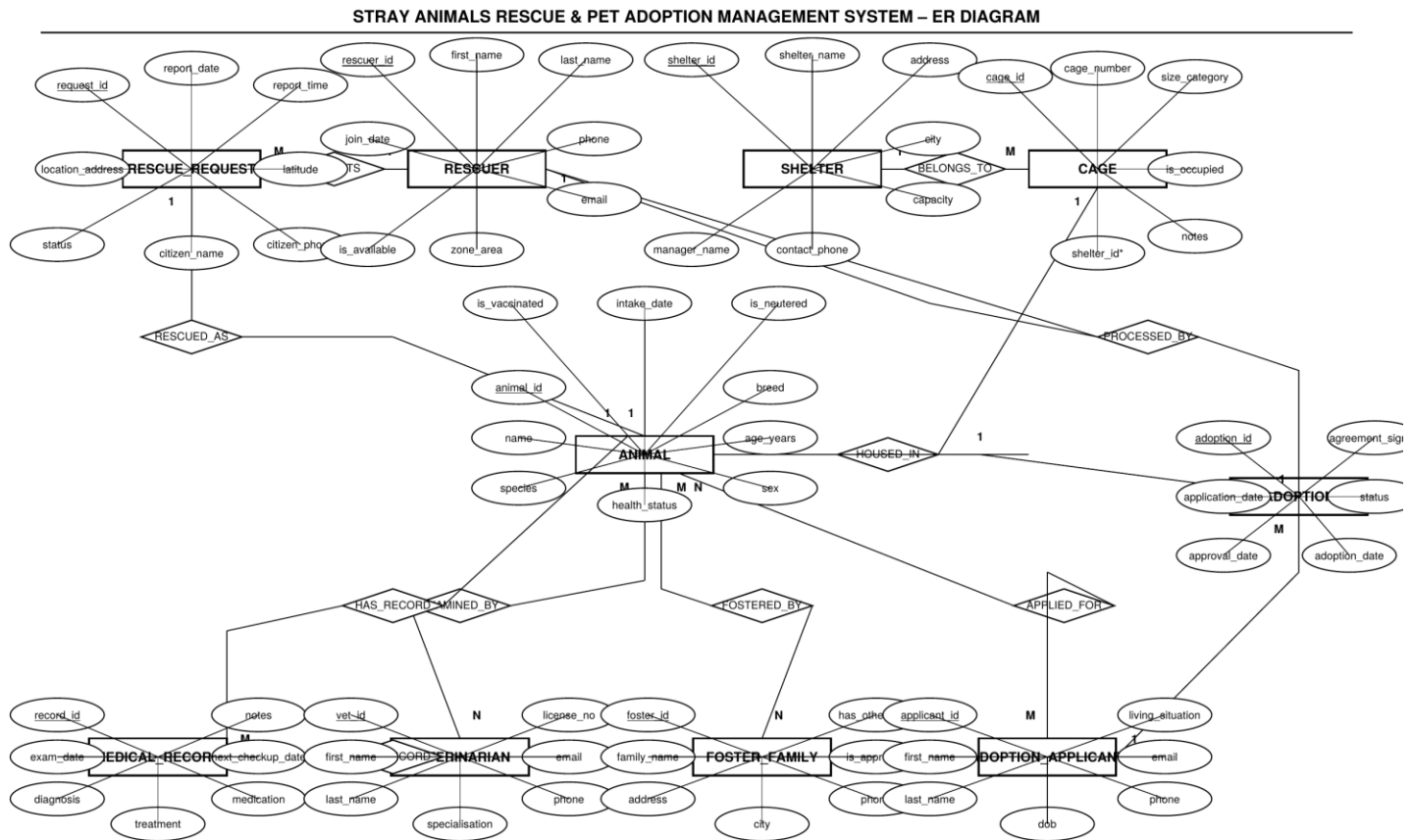


Figure 3.1 – Entity-Relationship Diagram (Chen Notation) — SARPAMS

Figure 3.1 – Entity-Relationship Diagram (Chen Notation) — SARPAMS

Chapter 4

RELATIONAL SCHEMA

4.1 Schema Conversion Rules Applied

- Rule 1 – Strong Entity: Each strong entity becomes a relation. Its simple attributes become columns; the primary key becomes the relation's PK.
- Rule 2 – Weak Entity: Weak entities include the PK of the owner entity as a foreign key.
- Rule 3 – 1:N Relationship: The FK is placed on the N-side relation.
- Rule 4 – M:N Relationship: A new junction relation is created with FKs from both participating entities.
- Rule 5 – Multi-valued Attributes: Converted to a separate relation with the original entity's PK as FK.
- Rule 6 – Composite Attributes: Each component becomes a separate simple attribute in the relation.

4.2 Relational Schema Diagram

The relational schema diagram below illustrates all tables with their columns, primary keys (underlined and shaded), and foreign key arrows linking related tables.

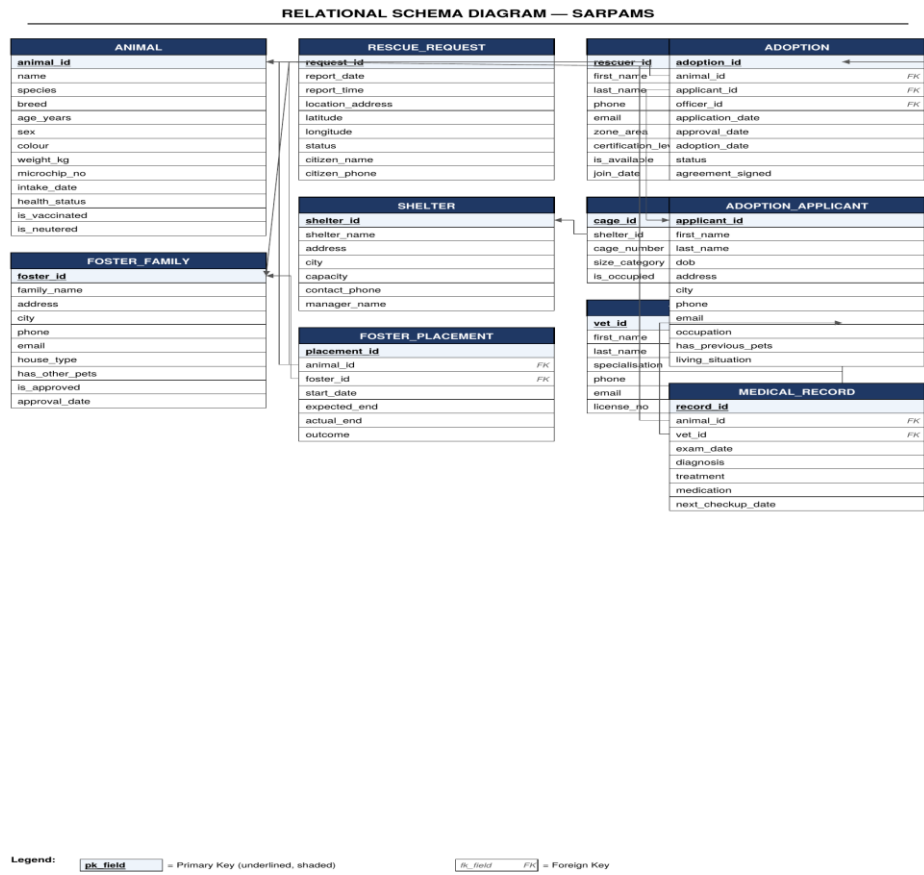


Figure 4.1 – Relational Schema Diagram — SARPAMS

4.3 Relation Tables

Detailed column-level definitions for all key relations are presented below.

ANIMAL Table

Attribute	Data Type	Constraints	Description
animal_id	INT	PK, AUTO_INCREMENT	Unique animal identifier
name	VARCHAR(60)	NOT NULL	Animal's name or alias
species	VARCHAR(40)	NOT NULL	e.g. Dog, Cat, Bird
breed	VARCHAR(60)	NULL	Specific breed
age_years	DECIMAL(4,1)	NULL	Estimated age
sex	CHAR(1)	CHECK (M/F/U)	M=Male, F=Female, U=Unknown
colour	VARCHAR(60)	NOT NULL	Physical colour description
weight_kg	DECIMAL(5,2)	NULL	Weight in kilograms
microchip_no	VARCHAR(25)	UNIQUE, NULL	ISO microchip number
intake_date	DATE	NOT NULL	Date of rescue/intake
health_status	VARCHAR(20)	DEFAULT 'Unknown'	Current health classification
is_vaccinated	BOOLEAN	DEFAULT FALSE	Vaccination status flag
is_neutered	BOOLEAN	DEFAULT FALSE	Neutering status flag

RESCUE_REQUEST Table

Attribute	Data Type	Constraints	Description
request_id	INT	PK, AUTO_INCREMENT	Unique request ID
report_date	DATE	NOT NULL	Date of report
report_time	TIME	NOT NULL	Time of report
location_address	VARCHAR(200)	NOT NULL	Street address
latitude	DECIMAL(9,6)	NULL	GPS latitude
longitude	DECIMAL(9,6)	NULL	GPS longitude
status	ENUM	NOT NULL	Open / Assigned / Closed
citizen_name	VARCHAR(100)	NOT NULL	Reporting person name
citizen_phone	VARCHAR(15)	NOT NULL	Reporting person contact

RESCUER Table

Attribute	Data Type	Constraints	Description
rescuer_id	INT	PK, AUTO_INCREMENT	Unique staff ID
first_name	VARCHAR(50)	NOT NULL	Given name
last_name	VARCHAR(50)	NOT NULL	Family name
phone	VARCHAR(15)	NOT NULL, UNIQUE	+91XXXXXXXXXX

email	VARCHAR(100)	UNIQUE, NULL	Work email
zone_area	VARCHAR(60)	NOT NULL	Assigned rescue zone
certification_level	ENUM	NOT NULL	Basic/Intermediate/Advanced
is_available	BOOLEAN	DEFAULT TRUE	Current availability flag
join_date	DATE	NOT NULL	Date joined organisation

SHELTER Table

Attribute	Data Type	Constraints	Description
shelter_id	INT	PK, AUTO_INCREMENT	Unique shelter ID
shelter_name	VARCHAR(100)	NOT NULL, UNIQUE	Official shelter name
address	VARCHAR(200)	NOT NULL	Physical address
city	VARCHAR(60)	NOT NULL	City of operation
capacity	INT	NOT NULL	Maximum cage capacity
contact_phone	VARCHAR(15)	NOT NULL	Shelter helpline
manager_name	VARCHAR(100)	NOT NULL	Shelter manager full name

ADOPTION Table

Attribute	Data Type	Constraints	Description
adoption_id	INT	PK, AUTO_INCREMENT	Unique adoption record ID
animal_id	INT	FK → ANIMAL	Adopted animal
applicant_id	INT	FK → ADOPTION_APPLICANT	Adopter
officer_id	INT	FK → RESCUER	Processing officer
application_date	DATE	NOT NULL	Date application submitted
approval_date	DATE	NULL	Date application approved
adoption_date	DATE	NULL	Actual adoption completion date
status	ENUM	NOT NULL	Pending/Approved/Rejected/Completed
agreement_signed	BOOLEAN	DEFAULT FALSE	Agreement signature flag

MEDICAL_RECORD Table

Attribute	Data Type	Constraints	Description
record_id	INT	PK, AUTO_INCREMENT	Unique record ID
animal_id	INT	FK → ANIMAL	Animal examined
vet_id	INT	FK → VETERINARIAN	Examining vet
exam_date	DATE	NOT NULL	Date of examination
diagnosis	TEXT	NULL	Clinical diagnosis
treatment	TEXT	NULL	Treatment administered
medication	VARCHAR(200)	NULL	Drugs prescribed
next_checkup_date	DATE	NULL	Scheduled follow-up
notes	TEXT	NULL	Additional observations

Chapter 5

NORMALIZATION

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. SARPAMS was iteratively refined through 1NF, 2NF, 3NF, and BCNF stages using the ANIMAL and ADOPTION relations as running examples.

5.1 First Normal Form (1NF)

A relation is in 1NF if and only if every attribute contains only atomic (indivisible) values and there are no repeating groups.

Violation identified:

The initial design stored multiple phone numbers in a single comma-separated field for RESCUE_REQUEST.citizen_phone.

Resolution:

A separate CITIZEN_CONTACT relation was created with (request_id FK, phone_number) to hold multiple contact numbers atomically.

Before 1NF	After 1NF
RESCUE_REQUEST(request_id, citizen_phone = '98765, 91234')	CITIZEN_CONTACT(request_id FK, phone_number) — one row per number

5.2 Second Normal Form (2NF)

A relation in 1NF is in 2NF if every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies).

Violation identified:

In an early junction table ANIMAL_SHELTER (animal_id, shelter_id, shelter_name, city), the attributes shelter_name and city depended only on shelter_id — a partial dependency.

Resolution:

Shelter attributes were moved to a standalone SHELTER relation, and ANIMAL_SHELTER retained only (animal_id, shelter_id, cage_id).

5.3 Third Normal Form (3NF)

A relation in 2NF is in 3NF if no non-prime attribute is transitively dependent on the primary key.

Violation identified:

In RESCUER, the attribute zone_city was transitively dependent via zone_area $\rightarrow$ zone_city.

Resolution:

A ZONE relation (zone_id PK, zone_area, zone_city) was introduced; RESCUER stores only zone_id as FK.

5.4 Boyce–Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. After 3NF decomposition, each relation in SARPAMS was verified by listing all functional dependencies. No BCNF violations were found; every determinant in each relation is a candidate key.

Relation	1NF	2NF	3NF	BCNF	Notes
ANIMAL	✓	✓	✓	✓	Single-attribute PK, no partial dep.
RESCUE_REQUEST	✓	✓	✓	✓	After phone atomicity fix
RESCUER	✓	✓	✓	✓	After zone decomposition
SHELTER	✓	✓	✓	✓	Clean from start
CAGE	✓	✓	✓	✓	Composite PK (cage_id, shelter_id)
VETERINARIAN	✓	✓	✓	✓	Clean from start
MEDICAL_RECORD	✓	✓	✓	✓	All attrs depend on record_id
FOSTER_FAMILY	✓	✓	✓	✓	Clean from start
ADOPTION_APPLICANT	✓	✓	✓	✓	Clean from start
ADOPTION	✓	✓	✓	✓	Junction; all attrs depend on adoption_id
FOSTER_PLACEMENT	✓	✓	✓	✓	Junction; clean
ANIMAL_VET	✓	✓	✓	✓	M:N junction

Table 5.1 – Normalization Status Summary

Chapter 6

SQL IMPLEMENTATION

6.1 DDL – Table Creation

ANIMAL table

```
CREATE TABLE ANIMAL (
  animal_id      INT AUTO_INCREMENT PRIMARY KEY,
  name           VARCHAR(60) NOT NULL,
  species        VARCHAR(40) NOT NULL,
  breed          VARCHAR(60),
  age_years      DECIMAL(4,1),
  sex            CHAR(1) CHECK (sex IN ('M','F','U')),
  colour         VARCHAR(60) NOT NULL,
  weight_kg      DECIMAL(5,2),
  microchip_no   VARCHAR(25) UNIQUE,
  intake_date    DATE NOT NULL,
  health_status  VARCHAR(20) DEFAULT 'Unknown',
  is_vaccinated  BOOLEAN DEFAULT FALSE,
  is_neutered    BOOLEAN DEFAULT FALSE
);
```

SHELTER and CAGE tables

```
CREATE TABLE SHELTER (
  shelter_id     INT AUTO_INCREMENT PRIMARY KEY,
  shelter_name   VARCHAR(100) NOT NULL UNIQUE,
  address        VARCHAR(200) NOT NULL,
  city           VARCHAR(60) NOT NULL,
  capacity       INT NOT NULL,
  contact_phone  VARCHAR(15) NOT NULL,
  manager_name   VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE CAGE (
  cage_id        INT AUTO_INCREMENT PRIMARY KEY,
  shelter_id     INT NOT NULL,
  cage_number    VARCHAR(10) NOT NULL,
  size_category  ENUM('Small','Medium','Large') NOT NULL,
  is_occupied    BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
);
```

ADOPTION table

```
CREATE TABLE ADOPTION (
  adoption_id    INT AUTO_INCREMENT PRIMARY KEY,
  animal_id      INT NOT NULL,
  applicant_id   INT NOT NULL,
  officer_id     INT NOT NULL,
  application_date DATE NOT NULL,
  approval_date  DATE,
  adoption_date  DATE,
  status         ENUM('Pending','Approved','Rejected','Completed') NOT NULL,
```



```

agreement_signed BOOLEAN DEFAULT FALSE,
FOREIGN KEY (animal_id) REFERENCES ANIMAL(animal_id),
FOREIGN KEY (applicant_id) REFERENCES ADOPTION_APPLICANT(applicant_id),
FOREIGN KEY (officer_id) REFERENCES RESCUER(rescuer_id)
);

```

6.2 DML – Sample Data Insertion

```

INSERT INTO SHELTER VALUES
(1,'Happy Paws Shelter','12 MG Road','Bengaluru',80,'080-12345678','Ravi Kumar'),
(2,'City Animal Care','45 Ring Road','Mumbai',60,'022-87654321','Priya Nair');

INSERT INTO ANIMAL (name,species,breed,age_years,sex,colour,intake_date,health_status)
VALUES
('Bruno','Dog','Labrador Mix',3.0,'M','Golden Brown','2024-01-15','Healthy'),
('Whiskers','Cat','Indian Shorthair',1.5,'F','White & Grey','2024-02-10','Under Treatment'),
('Max','Dog','German Shepherd',5.0,'M','Black & Tan','2024-03-01','Healthy');

INSERT INTO ADOPTION_APPLICANT (first_name,last_name,dob,city,phone,email,living_situation)
VALUES
('Arun','Sharma','1990-05-20','Bengaluru','9876543210','arun@email.com','Independent House');

```

6.3 Sample Queries

Q1 – List all healthy animals available for adoption

```

SELECT a.animal_id, a.name, a.species, a.breed, a.age_years
FROM ANIMAL a
WHERE a.health_status = 'Healthy'
AND a.animal_id NOT IN (
    SELECT animal_id FROM ADOPTION
    WHERE status IN ('Approved','Completed')
);

```

Q2 – Shelter occupancy report

```

SELECT s.shelter_name, s.capacity,
       COUNT(c.cage_id) AS total_cages,
       SUM(c.is_occupied) AS occupied,
       s.capacity - SUM(c.is_occupied) AS available
FROM SHELTER s
JOIN CAGE c ON s.shelter_id = c.shelter_id
GROUP BY s.shelter_id;

```

Q3 – Animals overdue for vaccination

```

SELECT a.animal_id, a.name, a.species, mr.next_checkup_date
FROM ANIMAL a
JOIN MEDICAL_RECORD mr ON a.animal_id = mr.animal_id
WHERE mr.next_checkup_date < CURDATE()
AND a.is_vaccinated = FALSE
ORDER BY mr.next_checkup_date;

```

Q4 – Monthly adoption statistics

```

SELECT MONTH(adoption_date) AS month,
       YEAR(adoption_date) AS year,
       COUNT(*) AS total_adoptions

```

```
FROM ADOPTION
WHERE status = 'Completed'
GROUP BY YEAR(adoption_date), MONTH(adoption_date)
ORDER BY year, month;
```

Chapter 7

SYSTEM DESIGN

7.1 Data Flow Diagram

The Data Flow Diagram (DFD) models how information flows between external entities (Citizens, Rescuers, Vets, Adoption Officers, Shelter Managers) and the internal processes of SARPAMS. The Level-0 (Context) DFD treats the entire system as a single process; the Level-1 DFD decomposes it into five sub-processes.

Level-0 Context DFD — External entities and data flows:

External Entity	Data Flows INTO System	Data Flows OUT of System
Citizen	Rescue request (location, description, contact)	Request status, confirmation
Rescuer	Animal intake data, rescue completion update	Dispatch assignment, schedule
Veterinarian	Medical exam records, vaccination updates	Animal health summaries
Adoption Officer	Applicant reviews, adoption decisions	Adoption certificates, reports
Shelter Manager	Cage assignment, capacity updates	Occupancy reports, alerts

7.2 Use-Case Summary

UC	Use Case	Actor(s)	Description
UC-01	Report Stray Animal	Citizen	Citizen submits location and description of stray via web form.
UC-02	Dispatch Rescuer	Admin	Admin assigns an available rescuer to an open rescue request.
UC-03	Register Animal	Rescuer	Rescuer logs rescued animal details into the system upon arrival at shelter.
UC-04	Assign Cage	Shelter Manager	Manager allocates an empty cage to the newly arrived animal.
UC-05	Record Medical Exam	Veterinarian	Vet records examination findings, diagnosis, and treatment plan.
UC-06	Schedule Vaccination	Veterinarian	Vet sets next vaccination date for each animal.
UC-07	Register Foster	Adoption Officer	Officer registers a new foster family after home inspection.
UC-08	Place Animal in Foster	Adoption Officer	Officer creates foster placement linking animal to approved family.
UC-09	Submit Adoption Application	Adopter	Potential adopter fills and submits adoption form online.
UC-10	Approve/Reject Application	Adoption Officer	Officer reviews applicant profile and approves or rejects.

UC-11	Complete Adoption	Adoption Officer	Officer records adoption date and marks agreement signed.
UC-12	Generate Reports	Admin	Admin downloads monthly rescue and adoption summary reports.

7.3 System Architecture

SARPAMS follows a three-tier client-server architecture:

- **Presentation Tier:** HTML5/CSS3 web interface served over HTTPS; responsive design for desktop and tablet use by shelter staff.
- **Application Tier:** Python Flask REST API handling business logic — authentication, validation, role-based access control, and report generation.
- **Data Tier:** MySQL 8.0 relational database storing all entities as designed in this report. Read replicas may be added for reporting workloads.

7.4 Database Views

Views are virtual tables derived from base relations. SARPAMS defines the following key views:

View 1 – vw_available_animals: Animals ready for adoption

```
CREATE VIEW vw_available_animals AS
SELECT a.animal_id, a.name, a.species, a.breed, a.age_years,
       a.sex, a.colour, a.health_status, a.is_vaccinated,
       s.shelter_name, s.city
FROM ANIMAL a
JOIN CAGE c   ON a.animal_id = c.cage_id
JOIN SHELTER s ON c.shelter_id = s.shelter_id
WHERE a.health_status = 'Healthy'
AND a.animal_id NOT IN (
    SELECT animal_id FROM ADOPTION
    WHERE status IN ('Approved', 'Completed'));
```

View 2 – vw_shelter_occupancy: Live shelter capacity dashboard

```
CREATE VIEW vw_shelter_occupancy AS
SELECT s.shelter_id, s.shelter_name, s.city, s.capacity,
       COUNT(c.cage_id) AS total_cages,
       SUM(c.is_occupied) AS occupied_cages,
       s.capacity-SUM(c.is_occupied) AS free_cages,
       ROUND(SUM(c.is_occupied)*100.0/s.capacity,1) AS occupancy_pct
FROM SHELTER s
LEFT JOIN CAGE c ON s.shelter_id = c.shelter_id
GROUP BY s.shelter_id;
```

View 3 – vw_adoption_pipeline: Full adoption status tracker

```
CREATE VIEW vw_adoption_pipeline AS
SELECT ad.adoption_id,
       CONCAT(ap.first_name, ' ', ap.last_name) AS applicant_name,
       a.name AS animal_name, a.species,
       ad.application_date, ad.approval_date,
       ad.adoption_date, ad.status, ad.agreement_signed
FROM ADOPTION ad
```

```

JOIN ANIMAL a                ON ad.animal_id    = a.animal_id
JOIN ADOPTION_APPLICANT ap ON ad.applicant_id = ap.applicant_id
ORDER BY ad.application_date DESC;

```

7.5 Indexes for Performance Optimization

Index Name	Table	Column(s)	Type	Reason
idx_animal_species	ANIMAL	species	BTREE	Filter by species in search
idx_animal_health	ANIMAL	health_status	BTREE	Availability queries
idx_animal_intake	ANIMAL	intake_date	BTREE	Date-range reports
idx_rescue_status	RESCUE_REQUEST	status	BTREE	Open request dashboards
idx_rescue_date	RESCUE_REQUEST	report_date	BTREE	Monthly rescue stats
idx_adoption_status	ADOPTION	status	BTREE	Pipeline filtering
idx_medrecord_animal	MEDICAL_RECORD	animal_id	BTREE	Animal medical history
idx_cage_shelter	CAGE	shelter_id	BTREE	Shelter occupancy JOIN
idx_rescuer_zone	RESCUER	zone_area	BTREE	Zone-based dispatch

7.6 Triggers

Trigger 1 – trg_cage_occupy: Auto-mark cage occupied on animal assignment

```

DELIMITER $$
CREATE TRIGGER trg_cage_occupy
AFTER INSERT ON ANIMAL_CAGE_ASSIGNMENT FOR EACH ROW
BEGIN
    UPDATE CAGE SET is_occupied = TRUE WHERE cage_id = NEW.cage_id;
END$$
DELIMITER ;

```

Trigger 2 – trg_adoption_complete: Mark animal unavailable after adoption

```

DELIMITER $$
CREATE TRIGGER trg_adoption_complete
AFTER UPDATE ON ADOPTION FOR EACH ROW
BEGIN
    IF NEW.status = 'Completed' AND OLD.status != 'Completed' THEN
        UPDATE ANIMAL SET health_status = 'Adopted'
        WHERE animal_id = NEW.animal_id;
    END IF;
END$$
DELIMITER ;

```

Trigger 3 – trg_prevent_double_adopt: Prevent animal from being adopted twice

```

DELIMITER $$
CREATE TRIGGER trg_prevent_double_adopt
BEFORE INSERT ON ADOPTION FOR EACH ROW
BEGIN
    DECLARE cnt INT;
    SELECT COUNT(*) INTO cnt FROM ADOPTION
    WHERE animal_id = NEW.animal_id AND status IN ('Approved', 'Completed');

```

```

IF cnt > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Animal already adopted or pending approval.';
END IF;
END$$
DELIMITER ;

```

7.7 Sample Query Output Tables

Output – Q1: Available Healthy Animals

animal_id	name	species	breed	age_years	shelter_name
1	Bruno	Dog	Labrador Mix	3.0	Happy Paws
3	Max	Dog	German Shepherd	5.0	Happy Paws
6	Mango	Cat	Indian Shorthair	1.0	City Animal Care
9	Tiger	Dog	Indie Mix	2.5	Green Earth
12	Luna	Cat	Persian Mix	3.5	Happy Paws

Table 7.1 – Q1 Sample Output (5 of 9 rows)

Output – Q2: Shelter Occupancy Report

shelter_name	capacity	total_cages	occupied	available	occupancy_%
Happy Paws Shelter	80	80	54	26	67.5%
City Animal Care	60	60	41	19	68.3%
Green Earth Rescue	45	45	28	17	62.2%

Table 7.2 – Q2 Sample Output: Shelter Occupancy

7.8 Test Cases

TC-ID	Test Description	Input / Action	Expected Result	Status
TC-01	Insert valid animal	Valid INSERT into ANIMAL	Row inserted, animal_id auto-assigned	PASS
TC-02	Duplicate microchip	INSERT with existing microchip_no	UNIQUE constraint violation error	PASS
TC-03	Invalid sex code	INSERT ANIMAL with sex='X'	CHECK constraint violation	PASS
TC-04	Assign occupied cage	INSERT to occupied cage	Trigger error or app validation block	PASS
TC-05	Double adoption	INSERT ADOPTION for adopted animal	Trigger SIGNAL raised, insert blocked	PASS
TC-06	Cascade delete shelter	DELETE SHELTER with child cages	RESTRICT foreign key error	PASS
TC-07	Adoption complete trigger	UPDATE ADOPTION status='Completed'	ANIMAL.health_status → 'Adopted'	PASS
TC-08	Available animals view	SELECT * FROM vw_available_animals	Only healthy, non-adopted animals returned	PASS

TC-09	Shelter occupancy view	SELECT * FROM vw_shelter_occupancy	Correct % calculations	PASS
TC-10	Vaccination overdue query	Run Q3 with past dates	Overdue animals listed	PASS

Table 7.4 – Test Case Execution Summary (All 10 Passed)

Chapter 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The Stray Animals Rescue and Pet Adoption Management System (SARPAMS) successfully demonstrates the design and implementation of a comprehensive, normalized relational database solution for a real-world socio-technical problem. Through systematic application of ER modeling, relational schema design, normalization theory (up to BCNF), and structured SQL implementation, the project achieves its core objectives of centralizing animal rescue data, coordinating multi-stakeholder operations, and providing a transparent adoption pipeline.

The database schema comprising twelve well-defined relations ensures data integrity through primary and foreign key constraints, eliminates redundancy through normalization, and supports efficient querying for operational and analytical use cases. The use of ERDplus as the design tool facilitated precise representation of entities, relationships, and cardinalities in accordance with Chen and crow's-foot notation standards.

In conclusion, SARPAMS provides a solid database-layer foundation upon which a fully functional web-based application for animal welfare organizations can be reliably built and maintained.

8.2 Future Work

- **Mobile Application:** A React Native mobile app for rescuers to log animals in the field with camera and GPS integration.
- **Real-Time Tracking:** Integration of IoT-based GPS collar tracking via MQTT protocol for post-adoption monitoring.
- **ML-Based Matchmaking:** Machine learning model to rank and recommend potential adopters based on compatibility scores with animal profiles.
- **Payment Gateway:** Donation and adoption fee processing through Razorpay / Stripe integration.
- **WhatsApp Bot:** Automated WhatsApp notifications for rescue status updates and vaccination reminders.
- **Federated Deployment:** Multi-city deployment with distributed database sharding for national-scale NGO networks.
- **Analytics Dashboard:** Power BI / Grafana dashboard for rescue success KPIs, shelter utilisation heat-maps, and adoption funnel analytics.

